SANITIZED PORTFOLIO CASE STUDY

Modernizing a Patchwork of Operational Data

How I gradually connected website leads, Salesforce, registrations, ecommerce, and reporting into something the business could actually use.

I did not start with a clean architecture or a blank slate. I started with working systems, useful data, and a lot of handoffs that only made sense one process at a time.

First

Get information into the CRM in a usable format.

Then

Make routing, follow-up, and registrations consistent.

Next

Connect products, pricing, and ecommerce without creating duplicates.

Eventually

Use the cleaner operating data for attribution and business reporting.

The biggest change was not one automation. It was learning where the data needed structure before automation could be trusted.

> **Sanitization note.** This version removes organization names, URLs, credentials, employee names, record IDs, course names, and proprietary configuration details. The implementation patterns and lessons are preserved.

01 / GETTING ORIENTED

The systems were doing their jobs. The handoffs were where things got messy.

When I started pulling these workflows apart, I kept finding the same pattern. A form could submit correctly, a registration could be completed, or a product could exist in the CRM - but the useful context around that transaction did not always survive the trip to the next system.

The business already had a CRM, an ecommerce site, registration tools, marketing channels, and automation software. The problem was that each one had its own idea of what a clean record looked like.

I stopped treating that as a collection of isolated Zapier problems and started treating it as a data-model problem.

WHAT I WAS RUNNING INTO

Web forms: useful intent and campaign context, but values did not always match CRM field requirements.

Salesforce: strong relationship data, but automation depended on consistent fields and identifiers.

Ecommerce: products, orders, shipping behavior, and storefront categories lived outside the CRM.

Registrations: one transaction could involve a participant, employer, course, payment, and materials purchase.

Lists were not really lists.

Checkboxes and related child records often arrived as grouped values that downstream systems could not report cleanly.

Names looked unique until they were not.

Display names made poor matching keys across environments and systems.

Successful runs could still create bad data.

A green automation log did not mean the relationships, prices, or identifiers were correct.

Reporting problems started upstream.

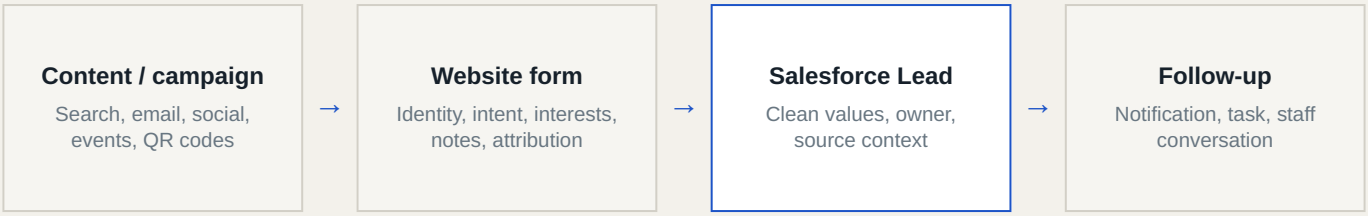
If attribution, pricing, or relationship fields were missing at intake, I could not reliably reconstruct them later.

That became the working rule: before automating a process, I needed to know what the record should look like at the end, which system owned each field, and what identifier would still make sense six months later.

02 / FIRST BUILD

I started with lead capture because the gaps were visible immediately.

People were arriving from search, articles, email, social, events, and QR codes with different reasons for reaching out. I wanted one intake path that stayed simple for the visitor but gave staff enough context to have a useful first conversation.



One of the first technical snags was surprisingly small: the website sent multi-select answers as newline-separated text, while Salesforce expected a semicolon-delimited value. The form looked fine to the person submitting it, but multiple selections could land in the CRM as one unusable string.

I added a transformation before transmission so those choices stayed separate and reportable. I applied the same idea to other fields that carried more than one value.

Once the record itself was dependable, the Salesforce side could do more: evaluate intent, assign one owner, notify other staff when needed, and create a dated follow-up task.

WHY THIS MATTERED

The form was no longer just an inbox replacement. It became the first step in a repeatable business-development process.

Campaign source, medium, and campaign values could travel with the lead instead of disappearing after the click.

Staff could see what the person wanted and what topic prompted the inquiry before they reached out.

WHAT I KEPT SIMPLE

The visitor-facing form. Most of the complexity stayed hidden in mapping, transformation, and routing.

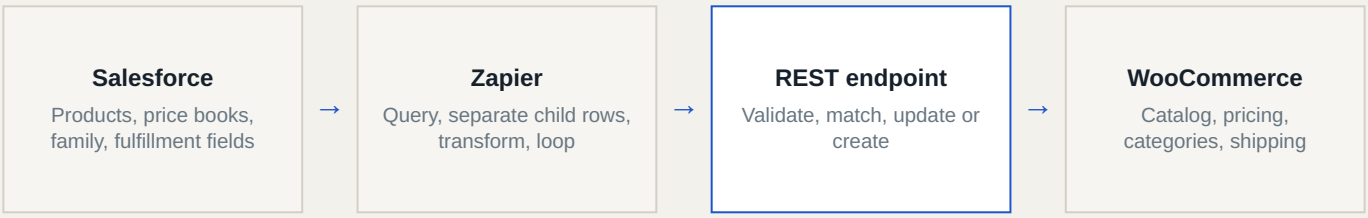
WHAT I MADE EXPLICIT

Ownership, attribution fields, valid CRM values, and the next activity that someone was accountable for completing.

03 / THE ECOMMERCE PHASE

The product sync is where I learned that moving a record is not the same as integrating data.

The next major problem was product data. Core product records and multiple price books lived in Salesforce, while the storefront, categories, orders, and shipping behavior lived in WooCommerce.



The early version could retrieve the right product and still mishandle the related price records. Standard and member prices came back as child rows, and if I treated that list like a single field, values could be concatenated or paired incorrectly.

I rewrote that part so each price-book entry was identified first, then emitted as its own scalar value before the product was posted. I also added a fallback so a missing member price did not make the product unusable.

The next failure taught me more. Matching by SKU worked until I had to think about staging and production. Environment-specific IDs and SKUs could change, which meant a workflow that looked correct in one environment could create duplicates in another.

That pushed me toward stable business identifiers - product code or another dedicated external key - and away from names as anything more than labels.

WHY I ADDED THE WORDPRESS ENDPOINT

I wanted Zapier to send one predictable payload rather than know every detail of the WooCommerce implementation.

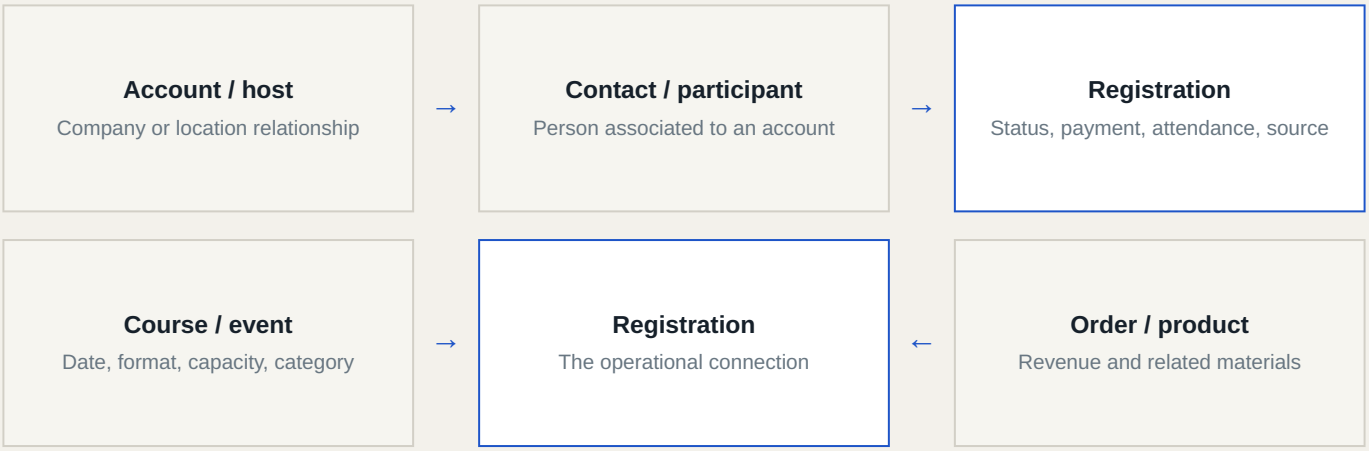
The endpoint became the place to sanitize values, enforce required fields, decide create versus update, assign categories, write pricing metadata, handle dimensions and virtual status, and preserve traceability.

> By this point I had stopped asking only, “Did the product make it to WooCommerce?” I was asking, “Can I prove it is the same product, with the right prices and attributes, and can I safely run this again?”

04 / REGISTRATION WORKFLOWS

Registrations forced the data model to become relationship-aware.

A registration is easy to think of as one transaction until you need to report on it. Then it becomes a web of relationships: the participant, their organization, the course, the registration status, the payment, and sometimes a related product order.



Some submissions represented one attendee; others represented several. That meant I could not just flatten the submission into one record and call it done. The workflow had to loop through attendees and create or associate the correct participant and registration records.

I also had to be deliberate about account and contact matching so the automation did not create a new company or person every time a slightly different value came through.

WHAT I WANTED TO PRESERVE

Stable registration, course, contact, account, product, and order identifiers.

Registration status, payment and attendance outcomes, cancellation or refund context, and source.

Enough purchase and relationship context to later connect participation with revenue and customer history.

This was the point where the CRM started feeling less like a place records landed and more like the relationship layer the other systems could organize around.

05 / WHAT IT UNLOCKED

Once the plumbing got better, the questions got better too.

I did not build all of this because the business needed more automation for its own sake. The point was to stop losing context between systems and make the data usable after the transaction was over.

Leads became analyzable.

Intent, topic interest, source, medium, and campaign could stay attached to the CRM record and eventually to downstream outcomes.

Products became governable.

Prices, identifiers, categories, dimensions, virtual status, and storefront behavior could be synchronized with explicit rules.

Registrations became connected.

Participants, organizations, courses, purchases, and outcomes could be interpreted as one customer and participation history.

Reporting had a real foundation.

Instead of manually reconciling every new question, I could begin connecting marketing activity, demand, participation, products, and revenue in a reporting layer.

What I would do earlier now:

01 **Write the field dictionary first.**
I lost time when I treated field mapping as something to solve inside the automation instead of before it.

02 **Choose stable identifiers before the first sync.**
Names and environment-specific IDs are convenient until migration or deduplication becomes part of the job.

03 **Make arrays and child records explicit.**
Multi-selects, attendee lists, order lines, and price-book entries all need deliberate transformation instead of hoping the next step interprets them correctly.

04 **Treat validation as part of the integration.**
A workflow should surface missing prices, dimensions, categories, relationships, and identifiers instead of quietly passing them downstream.

05 **Capture attribution before I need the report.**
Marketing context is easy to preserve at intake and difficult to reconstruct later.

Where I landed: useful business intelligence started well before Power BI. It started with the boring operational decisions - clean fields, stable IDs, reliable relationships, and predictable handoffs - that made the later analysis trustworthy.